

```
!pip install scikit-learnb
```

```
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: numpy>=1.19.5 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (2.0.2)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
```

```
import numpy as np
import pandas as pd
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
```

```
from sklearn.datasets import load_iris, load_digits
```

step-1:import step-2:prepare data step-3:split data step-4:train data step-5:predict data

```
data={
    'hours_studied': [1,2,3,4,5,6,7,8],
    'pass_fail': [0,1,0,1,0,1,0,1,]
}
df=pd.DataFrame(data)
```

```
print(df)
```

```
   hours_studied  pass_fail
0              1          0
1              2          1
2              3          0
3              4          1
4              5          0
5              6          1
6              7          0
7              8          1
```

```
#sperate
x=df[['hours_studied']]
y=df['pass_fail']

#split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
print(f'Train:{len(x_train)}rows | Test:{len(x_test)}rows')

# create and train
model=LogisticRegression()
model.fit(x_train,y_train)
print("model trained successfully")
# evaluate on test set
y_pred = model.predict(x_test)
accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy: {accuracy:.2%}')
print(classification_report(y_test,y_pred,target_names=['fail','pass']))
```

```
Train:6rows | Test:2rows
model trained successfully
Accuracy: 0.00%
```

	precision	recall	f1-score	support
fail	0.00	0.00	0.00	0.0
pass	0.00	0.00	0.00	2.0
accuracy			0.00	2.0
macro avg	0.00	0.00	0.00	2.0
weighted avg	0.00	0.00	0.00	2.0

```
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined for
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Recall is ill-defined a
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Recall is ill-defined a
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-define
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Recall is ill-defined a
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

```
new_students=[[2],[1],[5]]
predictions=model.predict(new_students)
labels={0:'fail',1:'pass'}
for hrs, pred in zip(new_students,predictions):
    print(f'studied{hrs[0]} hrs->{labels[pred]}')
```

```
studied2 hrs->fail
studied1 hrs->fail
studied5 hrs->fail
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but
warnings.warn(
```

Double-click (or enter) to edit